

[Sistema Inteligente de Control de Temperatura con Regulación PWM]

Docente: Diego Miranda Henríquez.

Integrantes: Mauricio Henríquez, Benjamín Lazo,

Gabriel Espinoza, Diego Allendes.

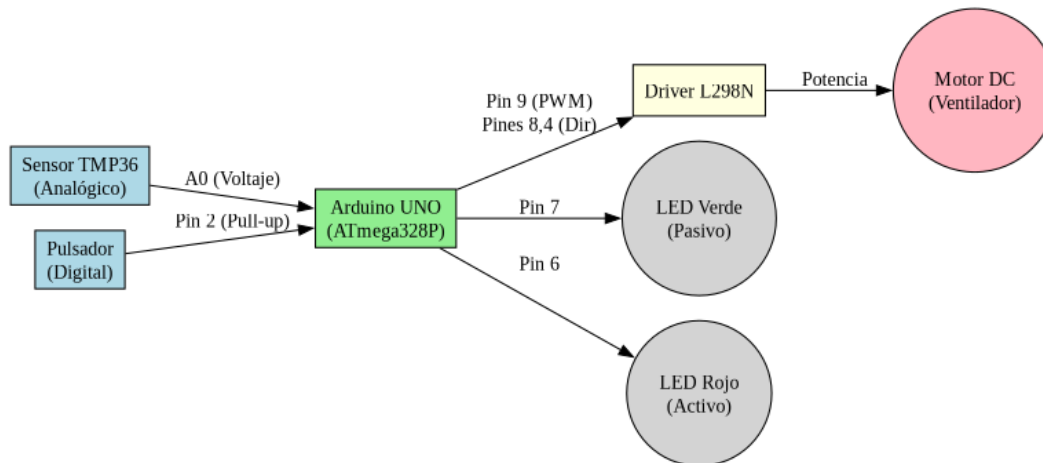
Asignatura: ISF-215 Hardware digital - Tarea 1.

Introducción.

- El presente informe tiene como objetivo principal consolidar el aprendizaje sobre el uso de los componentes electrónicos y el funcionamiento de un sistema embebido basado en Arduino UNO. A través del análisis del proyecto, se busca comprender cómo interactúan los sensores, actuadores y la lógica de control para regular un sistema de refrigeración de manera autónoma.

Diseño de Sistema.

Diagrama de Bloques del Sistema:



-El sistema se compone de tres bloques principales, orquestados por el microcontrolador.

1. Bloque de Entrada (Control y Contabilidad):

- a. **Sensor de Temperatura (TMP36):** Actúa como transductor, entregando un voltaje linealmente proporcional a la temperatura.
- b. **Pulsador Táctil:** Entrada digital que permite al usuario alternar entre el modo de control automático y el modo actual.

2. Bloque de procesamiento (Arduino UNO/ATMega328P):

Convierte señales analógicas a digitales mediante su *ADC* interno.

- a. Filtra lecturas ruidosas del pulsador mediante técnicas de *Debouncing* para software.
- b. Ejecuta el algoritmo de control lógico para determinar el ciclo de trabajo (*duty Cycle*) del *PWM*.

3. Bloques de Salida (Actuadores e interfaz HMI):

- a. **Driver de motor L298N + Motor DC:** El driver amplifica la señal *PWM* de control del Arduino para proporcionar la corriente necesaria que regula la velocidad del ventilador.
- b. **Controladores LED (verde y rojo):** Interfaz visual que comunica el estado del sistema. Verde para el estado “pasivo” (Ventilador apagado) y rojo para el estado “activo” (ventilador en movimiento).

Esquema de Conexiones.

Aunque en esta versión de texto se omite la representación gráfica, el diagrama físico (*Tinkercad*) establece las siguientes conexiones fundamentales:

- **Señales Analógicas:** El pin *Vout* el sensor *TMP36* se conecta al pin *A0* del Arduino.
- **Señales Digitales de Entrada:** El pulsador se conecta al pin 2. Se aprovecha la resistencia de *pull-up* interna del microcontrolador, conectando el otro lado del pulsador a *GND*.
- **Señales Digitales de Salida:** El ánodo del *LED* verde se conecta al pin 7 y el *LED* rojo se conecta al pin 6 (Ambos protegidos por resistencias de 220 OHM).
- **Control *PWM* y Potencia:** El pin 9 (capaz de generar *PWM* por hardware en el *ATmega328P*) se conecta al pin *ENA* del puente *H L298N* para controlar la velocidad. Los pines 8 y 4 del Arduino se conectan a *IN1* e *IN2* del driver para establecer el sentido del giro.

Descripción del Código.

El código base, desarrollado en C++, se estructura mediante técnicas de *Polling* continuo. El flujo principal dentro de la función `loop()` evalúa constantemente el estado del botón y la lectura del sensor analógico.

- **Manejo de la lectura analógica y conversión:** El *ADC* convierte el voltaje del sensor *TMP36* en un número entero. para relacionar este valor del *ADC* (0-1023) con una magnitud física real (°C), el código implementa la siguiente normalización:

```
int adc = analogRead(pinSensor);  
float voltaje = adc * 5.0 / 1023.0;  
float temperatura = (voltaje - 0.5) * 100.0;
```

- **Lógica de Control (*PWM*):** La función `calcularPWM(float temperatura)` definiendo el comportamiento del actuador. Existe una condición de umbral (`umbral = 30.0`). Si la temperatura es menor, el ventilador se detiene (Retorna `0`). Si se encuentra entre 30°C y 40°C, se realiza un escalamiento proporcional usando la función `map()` nativa de Arduino:

```
if (temperatura < 40) {  
    return map(temperatura, 30, 40, 90, 180);  
}
```

Si la temperatura supera los 40°C, el ventilador opera a máxima potencia (`255`).

Para asegurar una lectura limpia de la entrada digital (pulsador) sin recurrir a hardware adicional como capacitores, se implementó una función `manejarboton()` que evalúa si el estado de la señal se ha mantenido estable por un `tiempodebounce = 50` milisegundos utilizando la función de interrupción de tiempo `millis()`.

Análisis de Integración y Logros del Proyecto.

Este proyecto permitió completar con éxito un sistema integrado funcional que conecta el entorno físico con la lógica computacional. A nivel de adquisición de datos, se logró la lectura y estabilización de variables ambientales (temperatura) y entradas de usuario (pulsador) mediante técnicas de software como el Debouncing, evitando fluctuaciones mecánicas. A nivel de control y procesamiento, se implementó una lógica de estados estable, traduciendo correctamente la información de entrada en decisiones operativas. Finalmente, en la etapa de potencia y actuación, se concretó la modulación de energía (PWM) a través de un driver externo, logrando regular con precisión la velocidad de un actuador físico (motor DC) sin comprometer la integridad del microcontrolador.

Conclusiones.

A modo de cierre, este documento refleja el proceso de aprendizaje adquirido durante la exploración de sistemas digitales interactivos. La comprensión de la adquisición de señales analógicas, el control de potencia mediante PWM y el acondicionamiento de señales mediante software (Debouncing) constituyen conocimientos fundamentales asimilados que sientan las bases para el desarrollo de proyectos tecnológicos más complejos.